T4. Modelado de Comportamiento

Interacciones y mensajes

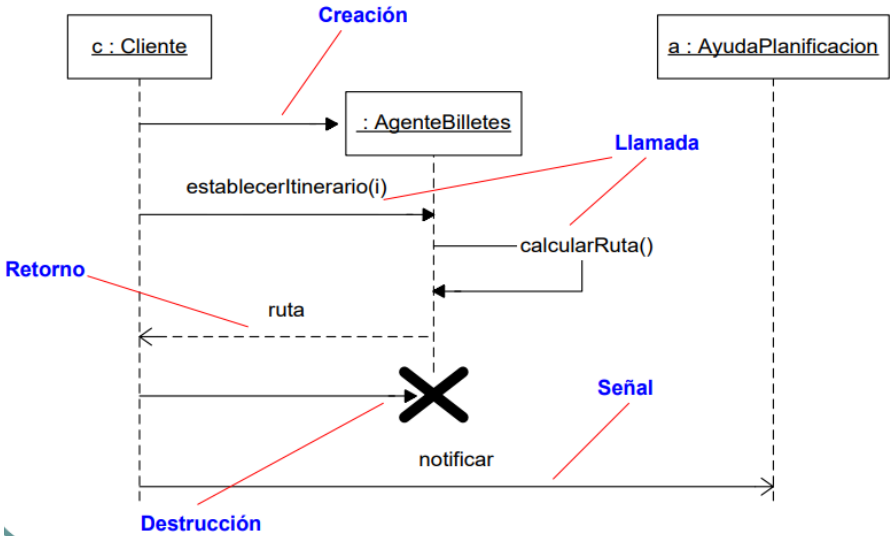
Una **interacción** es un conjunto de **mensajes** intercambiados por un grupo de objetos dentro de un contexto dado para lograr un **propósito** determinado. Es un modelado **dinámico** de **colaboraciones** (sociedad de objetos que proporcionan un comportamiento mayor que la suma de sus comportamientos).

Un **mensaje** es la transmisión de información entre objetos con una expectativa de respuesta, que desencadena una actividad. Tipos:

- **Llamada**: es una invocación de operaciones (mensajes de tipo síncrono, el emisor espera hasta que el receptor termine).
- **Retorno**: devuelven un valor o indicar finalización.
- Envío de **señales** (estímulos asíncronos, la respuesta puede tardar hasta que termine de ejecutar lo que estaba haciendo, no bloquea).
- **Creación** de objetos (en Java llamadas a constructor)
- **Destrucción** de objetos (permite decidir cómo se destruye un objeto).

Una **secuencia** es un flujo ordenado en el tiempo de mensajes encadenados entre diferentes objetos, se origina dentro de un **proceso** o **hilo** y continúa mientras ente exista. Cada proceso redefine un **flujo de control** separado en el que los mensajes se ordenan según se suceden en el tiempo.

Nota: Se trabaja con un único hilo de ejecución en nuestro caso.



En el diagrama aparecen tres objetos:

- c : Cliente (objeto con nombre 'c' de tipo Cliente).
- : AgenteBilletes (objeto anónimo de tipo AgenteBilletes).
- a : AyudaPlanificacion (objeto con nombre 'a').

Nota: Los objetos suelen ser **anónimos** salvo que haya instancias de la misma clase, se usa para diferenciarlas.

El objeto c:Cliente inicia la interacción (el objeto AgenteBilletes aún no existe). El Cliente envía un **mensaje** de **creación** a :AgenteBilletes, provocando que se instancie este objeto (comienza su línea de vida). Una vez creado, el Cliente envía un **mensaje síncrono** establecerItinerario(i) a AgenteBilletes, por lo que se invoca una operación del objeto y el Cliente queda **esperando** a que termine la ejecución (comportamiento síncrono). Durante la ejecución de establecerItinerario, el objeto AgenteBilletes realiza una **auto-llamada** síncrona a la operación calcularRuta(), por lo que el propio objeto utiliza sus métodos internos para completar la tarea y no intervienen otros objetos en este paso.

Una vez calculada la ruta, AgenteBilletes **devuelve** un **mensaje** de **retorno** con el valor ruta al Cliente, este retorno corresponde a la llamada anterior (es opcional en UML, pero si aparece debe estar asociado a una llamada previa). Tras completar su función, el objeto AgenteBilletes es destruido, lo que se representa con una "X" al final de su línea de vida (el objeto deja de existir en ese punto de la interacción, por lo tanto, es un objeto temporal/transitorio dentro de la secuencia). Finalmente, el Cliente envía un **mensaje** notificar al objeto a:AyudaPlanificacion, es una señal asíncrona por lo que el Cliente no espera respuesta (el objeto receptor procesará el mensaje cuando pueda).

Las interacciones sirven para modelar el **flujo de control** y la comunicación entre objetos en un sistema o un subsistema; así como la implementación de operaciones, clases o componentes. Los objetos en una interacción pueden ser tanto **concretos** como **prototípicos**, y pueden corresponder a instancas de clases **abstractas** o de **interfaces**.

Los **enlaces** son conexiones semánticas entre objetos por las que se transmite un mensaje. Normalmente, son instancias de una asociación. Existen adornos en los extremos de un enlace:

- **«association»**: indica que el enlace pertenece a una asociación.
- **«self»**: indica que el objeto se envía un mensaje a sí mismo (auto-enlace).
- **«global»**: accesible por tener alcance global.
- **«local»**: visible por estar declarado localmente.
- **«parameter»**: accesible por ser un parámetro.

Diagramas de Interacción

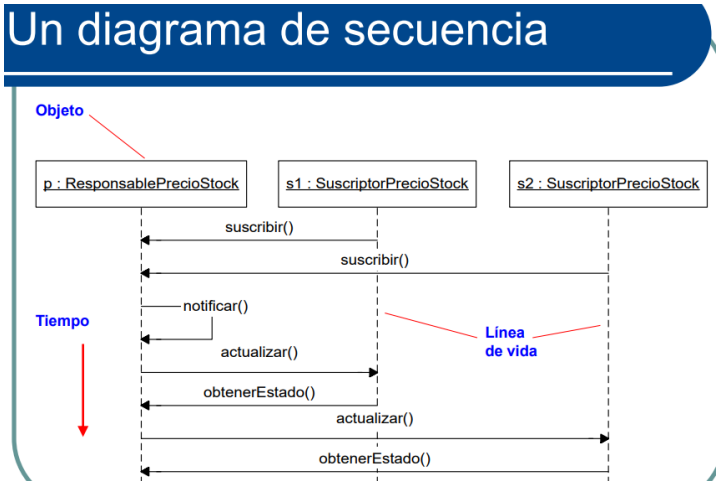
Los **diagramas de interacción** describen la forma en la que **colaboran** distintos objetos participantes para producir un comportamiento. Contienen objetos, enlaces y mensajes, y muestran la secuencia dinámica de mensajes que fluyen por los enlaces. Suelen capturar el comportamiento para **un solo caso de uso**. Hay dos tipos isomorfos/equivalentes:

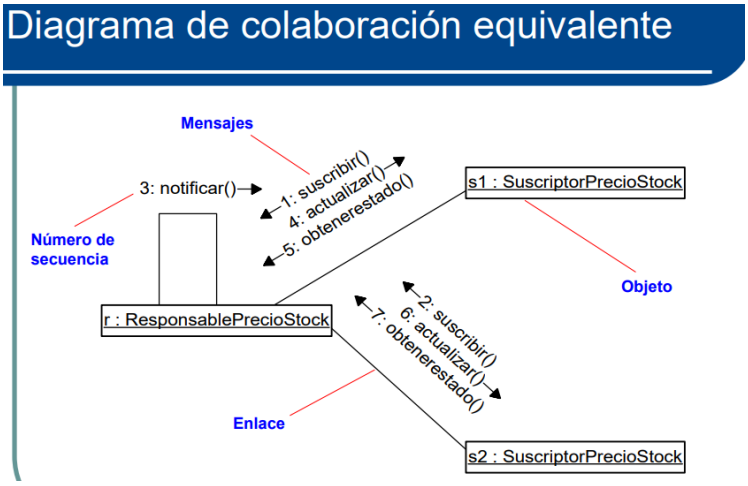
- **Diagramas de secuencia**: ordenaciones temporal de mensajes.
- **Diagramas de colaboración**: organización estructural de objetos.

Nota: Un diagrama de secuencia se puede transformar en un diagrama de colaboración y viceversa.

Los **diagramas de secuencia** tienen una **linea de vida** que representa la existencia de un objeto a lo largo de un período de tiempo y un **foco de control** que resalta el tiempo durante el cual un objeto ejecuta una acción ("está activo").

Las **diagramas de colaboración** tienen un **enlace** que se puede asociar a etiquetas especiales (**estereotipos**) en sus extremos que indican cómo se puede acceder al objeto (se puede omitir «association») y un **número de secuencia** que permite numerar los mensajes para indicar el **orden de ejecución** (numeración decimal de Dewey) y si hay llamadas dentro de otras (**anidamiento**).





El objeto p:ResponsablePrecioStock **inicia la interacción**, en este preciso momento los objetos s1 y s2 **existen** como participantes del sistema, pero la interacción comienza con el responsable. El objeto s1:SuscriptorPrecioStock envía un mensaje suscribir() al ResponsablePrecioStock, indicando que desea registrarse para recibir notificaciones de cambios de precio o stock. A continuación, s2:SuscriptorPrecioStock realiza la misma operación, enviando también un mensaje suscribir() al ResponsablePrecioStock (ambos quedan registrados como suscriptores del sistema).

En un momento posterior, el objeto p:ResponsablePrecioStock se **auto-envía** un mensaje notificar() como respuesta a un cambio (por ejemplo, modificación de precio o stock). Como consecuencia de la notificación, s1:SuscriptorPrecioStock recibe el mensaje actualizar() desde ResponsablePrecioStock (s1 es informado del cambio y debe actualizar su información). Durante el proceso de **actualización**, s1 solicita información adicional al responsable mediante el mensaje obtenerEstado(), este responde devolviendo el estado actual. A continuación, el objeto s2:SuscriptorPrecioStock también recibe el mensaje actualizar() enviado por ResponsablePrecioStock. De forma similar al caso anterior, s2 envía un

mensaje obtenerEstado() al responsable para obtener la información actualizada del sistema, este responde devolviendo el estado correspondiente.

Otro diagrama de secuencia

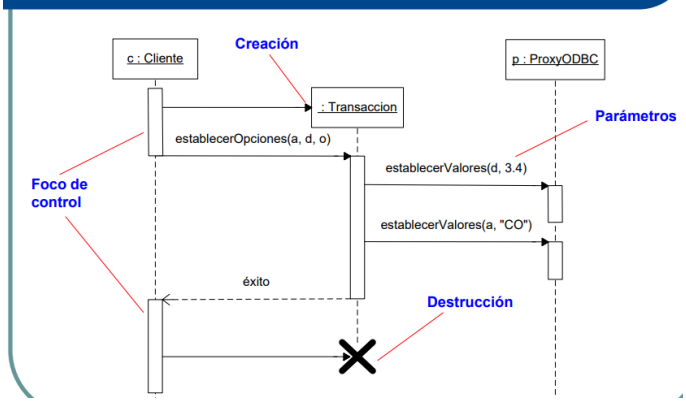
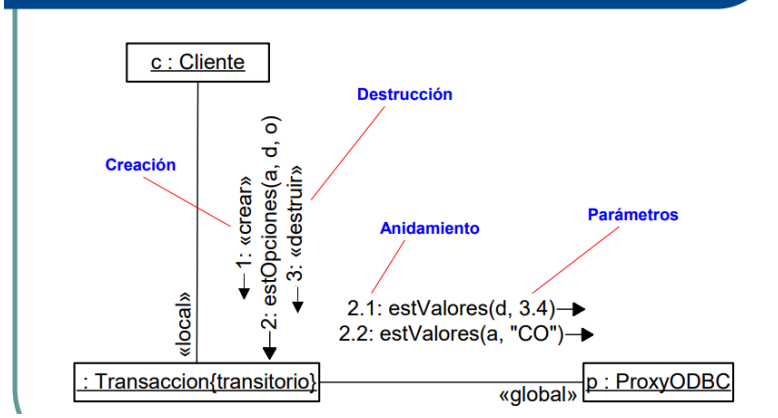


Diagrama de colaboración equivalente



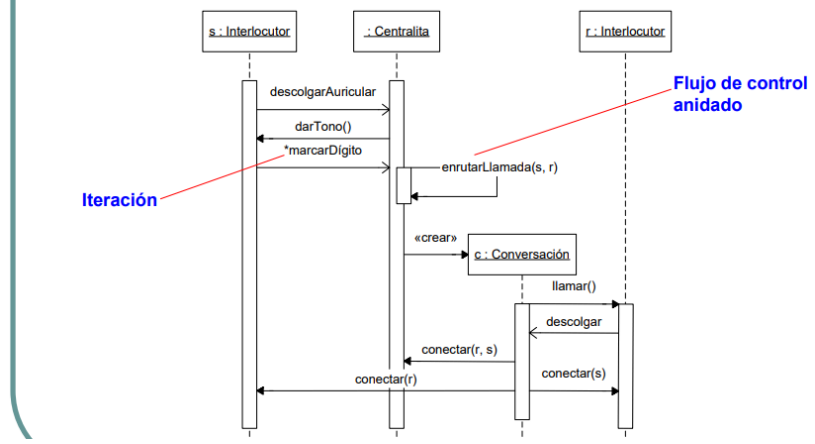
El objeto c:Cliente inicia la interacción. Primero, el Cliente envía un **mensaje de creación** a :Transaccion, provocando que se instancie (comienza su línea de vida). A continuación, el Cliente envía el **mensaje síncrono** establecerOpciones(a, d, o) a Transaccion, pasando una serie de parámetros necesarios para configurar la operación (como es síncrono, el Cliente **queda a la espera** de que termine la ejecución). Durante la ejecución de esta operación, el objeto Transaccion se comunica con p:ProxyODBC, envía el mensaje establecerValores(d, 3.4) y envía establecerValores(a, "CO"). Una vez completadas estas operaciones, el objeto Transaccion **devuelve** un resultado al Cliente mediante un mensaje de retorno (éxito), indicando que todo se ha ejecutado correctamente. Después de devolver el resultado, el objeto Transaccion es **destruido**, lo que se representa con una "X" al final de su línea de vida (indica que es un objeto temporal, creado solo para ejecutar esta operación concreta).

Iteración y bifurcación

Una **iteración** indica la repetición de un mensaje, se denota anteponiendo un * al mensaje.

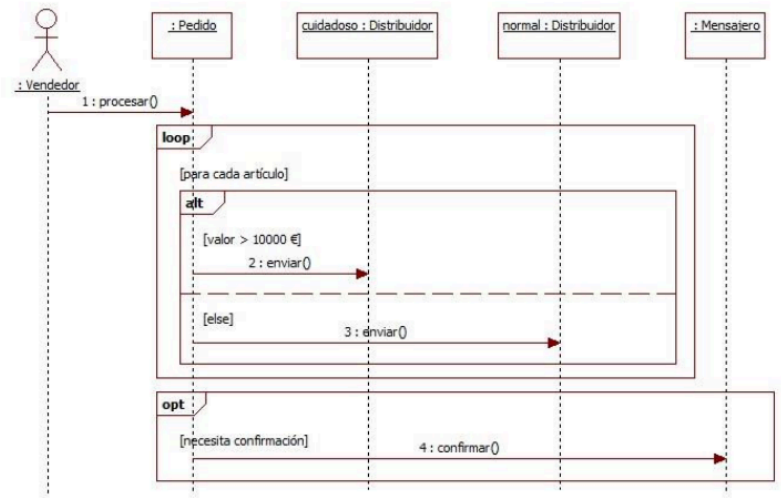
Una **bifurcación** representa un mensaje cuya ejecución depende de la evaluación de una condición, se denota adosando una cláusula de *guarda* entre corchetes (ej: [x>0]).

Otro diagrama de secuencia



El objeto s:Interlocutor inicia la interacción. Primero, s **envía** el **mensaje** descolgarAuricular a la Centralita, indicando que quiere iniciar una llamada. A continuación, la Centralita **responde** con darTono(), proporcionando señal de línea disponible. Después, s comienza a marcar el número mediante el mensaje marcarDigito, que se repite varias veces (bucle de marcado de dígitos). Una vez introducido el número completo, la Centralita ejecuta la operación enrutarLlamada(s, r), determinando el destinatario de la llamada.

Como resultado del enrutamiento, la Centralita crea el objeto c:Conversación mediante un **mensaje de creación** («crear»). El objeto c:Conversación inicia la comunicación con el receptor r:Interlocutor enviando el mensaje llamar(). El **receptor responde** con descolgar, aceptando la llamada. A continuación, se **establece la conexión**: Conversación envía conectar(s) a r y Centralita envía conectar(r) a s (ambos interlocutores quedan conectados y pueden comunicarse).



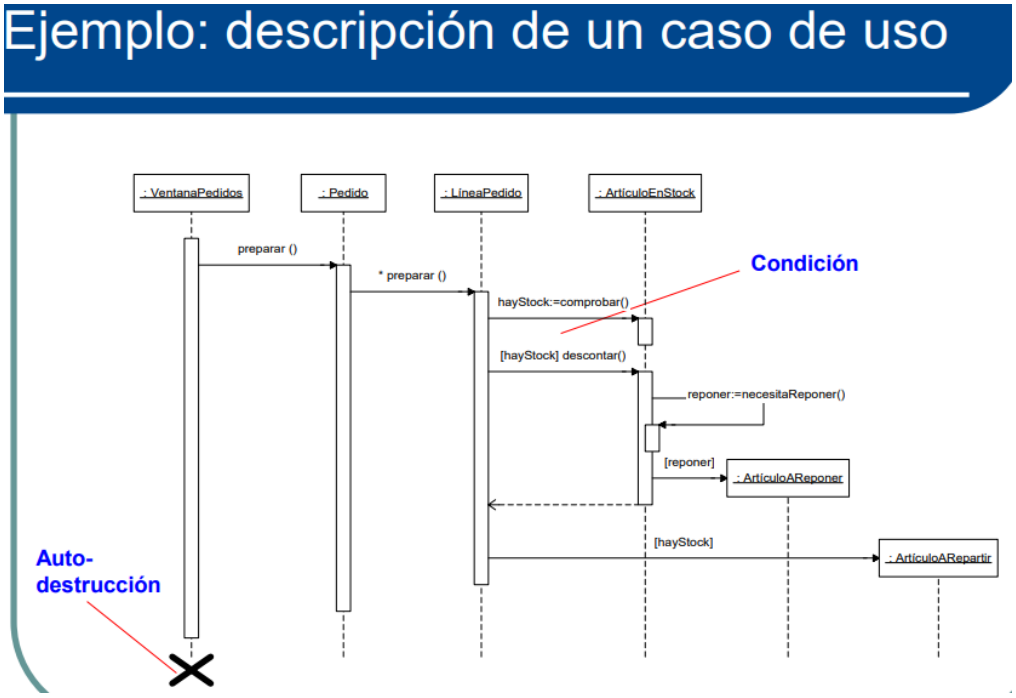
En UML 2 se introduce el concepto de **marcos (frames)** para estructurar los diagramas de secuencia. Un marco es un rectángulo que engloba una parte del

diagrama y permite indicar que los mensajes contenidos en su interior siguen un determinado **comportamiento** o **condición**. En la esquina superior izquierda del marco aparece una etiqueta que define su tipo.

Por ejemplo, el marco **loop** se utiliza para indicar iteraciones (while/for), es decir, que un conjunto de mensajes se repite varias veces. El marco **alt** representa alternativas, funcionando como una estructura if/else, donde se ejecuta un bloque u otro según se cumpla una condición. El marco **opt** indica un comportamiento opcional, equivalente a un if simple. Por otro lado, el marco **par** permite representar ejecuciones en paralelo, y el marco **ref** se utiliza para hacer referencia a otros diagramas, evitando repetir información.

Uso de los diagramas de iteracción y diferencias

Se usan para modelar colaboraciones y escenarios dentro de un caso de uso. Los diagramas de secuencia son preferibles para modelar escenarios o visualizar iteraciones y bifurcaciones sencillas, mientras que los diagramas de colaboración son para complejas.



El objeto :VentanaPedidos **inicia la interacción** enviando el mensaje preparar() al objeto :Pedido. A continuación, :Pedido **delega** la operación enviando el mensaje preparar() **iterativamente** al objeto :LineaPedido. El objeto :LineaPedido **consulta** la disponibilidad del producto enviando el mensaje comprobar() al objeto :ArticuloEnStock, obteniendo como resultado si hay stock disponible.

- Si hay stock, [hayStock] es true, se ejecuta el mensaje descontar() sobre :ArticuloEnStock, reduciendo la cantidad disponible.

Tras descontar el stock, el sistema comprueba si es necesario reponer mediante el mensaje necesitaReponer(). Si se cumple la condición [reponer], se genera una solicitud de reposición hacia el objeto :ArticuloAReponer. Independientemente de la reposición, si hay stock disponible [hayStock], se continúa el flujo enviando el producto al objeto :ArticuloARepartir, que se encargará de su distribución. Finalmente, el objeto :VentanaPedidos **finaliza su ciclo de vida** y se destruye automáticamente.

El **modelado en diagramas de secuencia** permite: establecer el **contexto** de la interacción, organizar **objetos** de izquierda a derecha (empezando por el que inicia la interacción), trazar las **líneas de vida** indicando creación y destrucción, colocar **mensajes** entre la líneas de vida de arriba hacia abajo, adornar con **focos de control** para mostral anidamiento o tiempo de computación, y usar varios diagramas para ver **bifurcación**.

La **ingeniería directa** es posible para ambos tipos de diagramas, en especial si el contexto es una operación. La **ingeniería inversa** también es factible, aunque puede generar demasiados detalles.

Comparación entre diagramas:

- Los **diagramas de interacción** sirven para mostrar colaboración entre varios objetos de un solo caso de uso.

- Los **diagramas de estados** sirven para mostrar comportamientos de un solo objeto a través de varios casos de uso.

- Los **diagramas de actividades** sirven para mostrar comportamiento a través de múltiples casos de uso o hilos.